

Ascend C 面向Tensor编程全新升级：Tensor API

作者 黄金华、吴兆麟、陈康

时间 2026/06/29

目录

Part 1 Tensor编程面临的挑战和痛点

Part 2 Tensor API: Tensor编程全新升级

Part 3 基于 Tensor API 的矩阵算子开发实践

Tensor编程面临的挑战和痛点

挑战与痛点

索引计算繁琐易错

传统开发中需手动计算多维**内存索引**，不仅代码冗长晦涩，还极易因边界处理失误引发难以调试的逻辑错误。

布局适配成本高昂

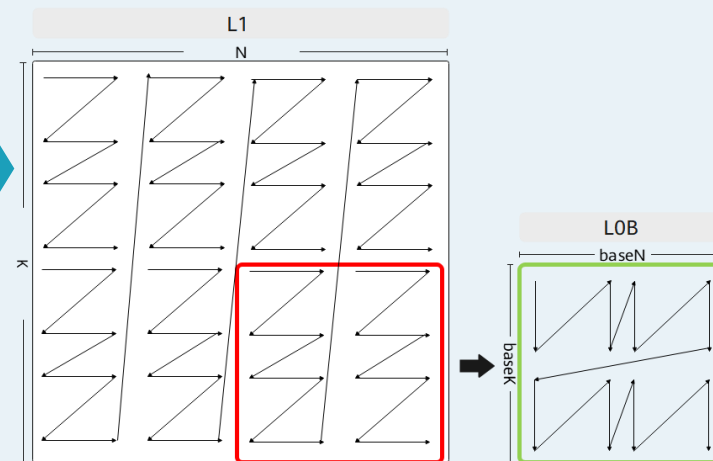
- ① 算法代码往往与行优先、列优先等**具体数据布局**强耦合，一旦更换硬件或调整数据排布，就需要重写大量核心代码。
- ② 数据布局（分形格式）存在**对齐、越界等约束**，手工校验复杂且容易遗漏出错。

挑战与痛点

示例

示例：L1->LOB 数据搬运索引计算复杂

```
AscendC::LoadData2DParams loadDataParams;  
uint32_t baseAddr = i * baseK * baseN + j * baseN * K;  
uint32_t dstOffset = curNAlign * CUBE_BLOCK;  
...  
for (int i = 0; i < AscendC::DivCeil(baseK,  
CUBE_BLOCK); ++i) {  
    AscendC::LoadData(l0B[i * dstOffset], l1B[srcAddr +  
baseAddr + i * srcOffset], loadDataParams);  
}
```



解决措施

目录

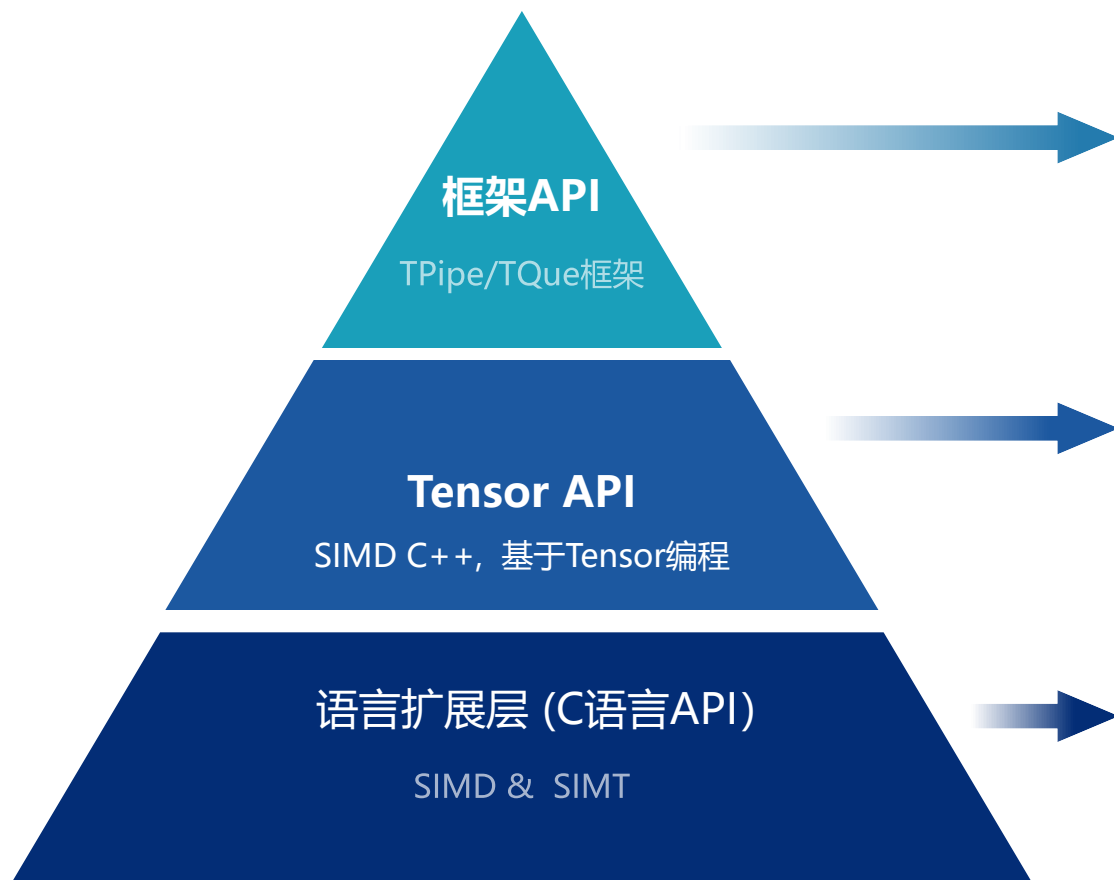
Part 1 传统Tensor编程面临的挑战和痛点

Part 2 Tensor API: Tensor编程全新升级

Part 3 基于 Tensor API 的矩阵算子开发实践

Ascend C 基于Tensor API 全面提升Tensor编程体验

基础API能力拆分为：C++ Tensor编程基础能力(Tensor API)、TPipe/TQue框架编程



01 | TPipe/TQue框架编程API

基于Tensor编程，通过TPipe/TQue自动管理内存搬运与同步，屏蔽底层细节；使用框架自动编排数据搬运和计算，提升编程易用性与开发效率

02 | 基于Tensor编程，提供C++完备编程能力

自主管理同步与内存布局，适配C++ Tensor开发习惯，开放全部底层硬件能力，支撑精细化调优与极致性能实现

03 | 基于指针编程，提供C完备编程能力

自主管理同步与内存，适配C语言开发习惯，开放全部底层硬件能力，支撑精细化调优与极致性能实现

Tensor API 设计理念：Layout升级为Tensor编程的一等公民

Layout

数据的逻辑蓝图

描述数据的逻辑形状和内存布局。它定义了数据“看起来”是什么样的，以及元素在内存中是如何排布的，是Tensor的“逻辑视图”。

Engine

数据的物理地址簿

描述数据的物理存储位置（如Global Memory、Local Memory）和访问方式。它定义了数据实际“在哪里”以及程序“如何”去访问这些数据。

Tensor = Engine (物理) + Layout (逻辑)

Tensor 是对数据的完整抽象，将物理存储与逻辑结构解耦，让开发者能专注于算法实现而非底层硬件细节。

Layout = Shape (形状) + Stride (步幅)

Shape 描述数据维度，Stride 描述内存中维度间的元素间隔，二者共同决定了数据在内存中的逻辑排布。

Tensor API 设计理念：兼顾性能与效率

零开销抽象，兼顾灵活性



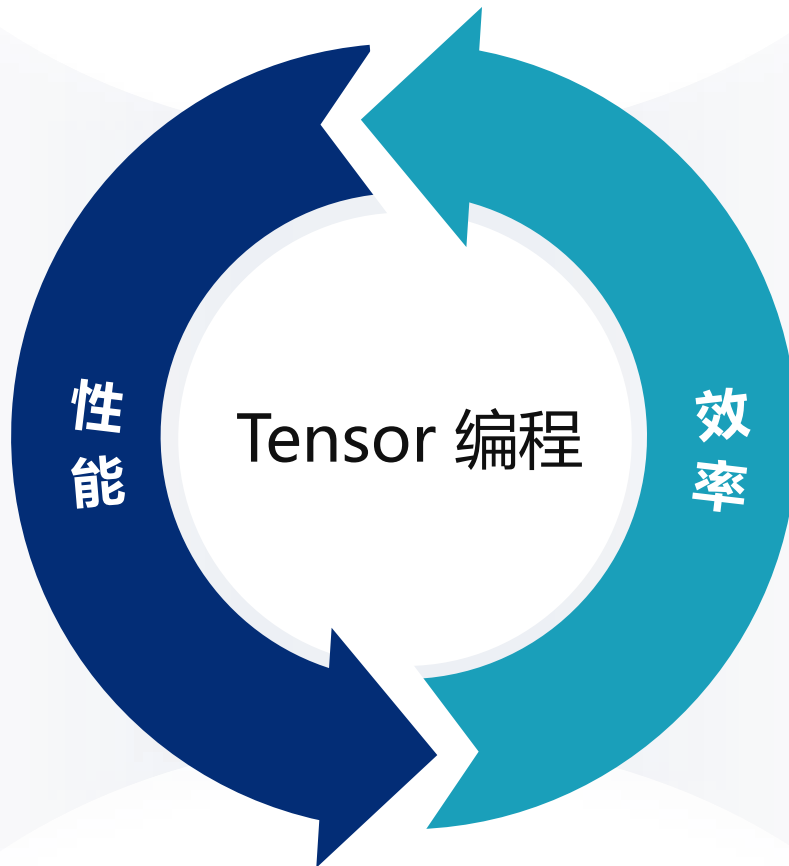
编译期极致优化

基于C++模板实现，所有静态信息在编译期完成计算与展开，运行时无任何额外开销，最终生成的机器码性能媲美纯手写底层代码。



动静结合的兼容性

兼容编译期静态尺寸与运行期动态尺寸，灵活适配从固定硬件参数到动态业务数据的各类复杂场景需求



布局代数，加速算子开发



可组合的数学体系

构建完整的布局代数运算体系，开发者可像搭积木一样，通过基础运算自由组合、变换数据布局，极大提升代码的复用性与灵活性。



显著降低开发门槛

无需深入底层硬件细节，即可快速设计、实现和迭代各类稠密线性代数运算，大幅缩短新算子的开发周期与学习成本。

Tensor API 基础数据类型和接口概述

Arch 接口

- ① 基于C API, **硬件语义的最小封装** (包括矩阵计算、矢量计算等) ;
- ② 增加Layout等元信息

Atom 接口 (原子计算)

- ① **指令的软件语义**, 由**Operation**、**Traits** 组成, 提供完整的编译元信息, 支撑上层的布局推导、组合拼装与静态正确性校验;
- ② 主要包括Te::CopyAtom、Te::MmadAtom等, 其中CopyAtom的Operation包含CopyGM2L1、CopyGM2UB等

Algorithm 接口

- ① 基于Tensor提供通用的算法原语接口

Tensor + Layout (以及Layout的代数运算)

Tensor API 核心概念介绍(1): Layout 概念解析

Shape (形状)

✓ 定义

定义多维对象的逻辑维度与大小，本质是整数元组

✓ 示例

Shape(4, 3)即代表一个4行3列的二维矩阵，描述了数据的逻辑组织结构

Stride (步长)

✓ 定义

定义维度坐标每加1时，内存地址需跨越的元素个数

✓ 示例

Stride(3, 1)，决定了数据在物理内存中的连续存储方式

Layout (布局)

✓ 定义

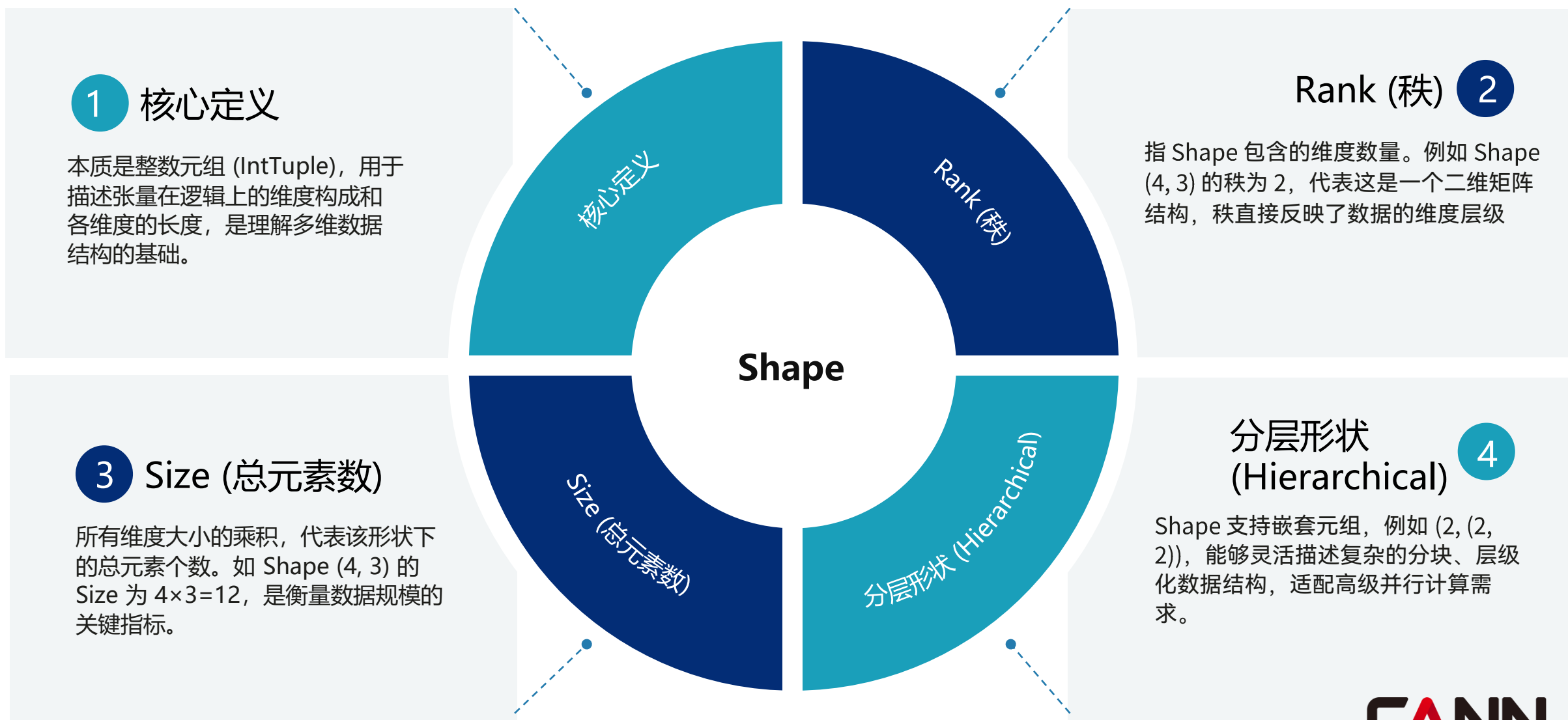
Shape 与 Stride 的组合函数对象，负责将多维逻辑坐标映射为一维内存索引。它像一本“地址字典”，精准定位每个逻辑元素在物理内存中的具体位置。



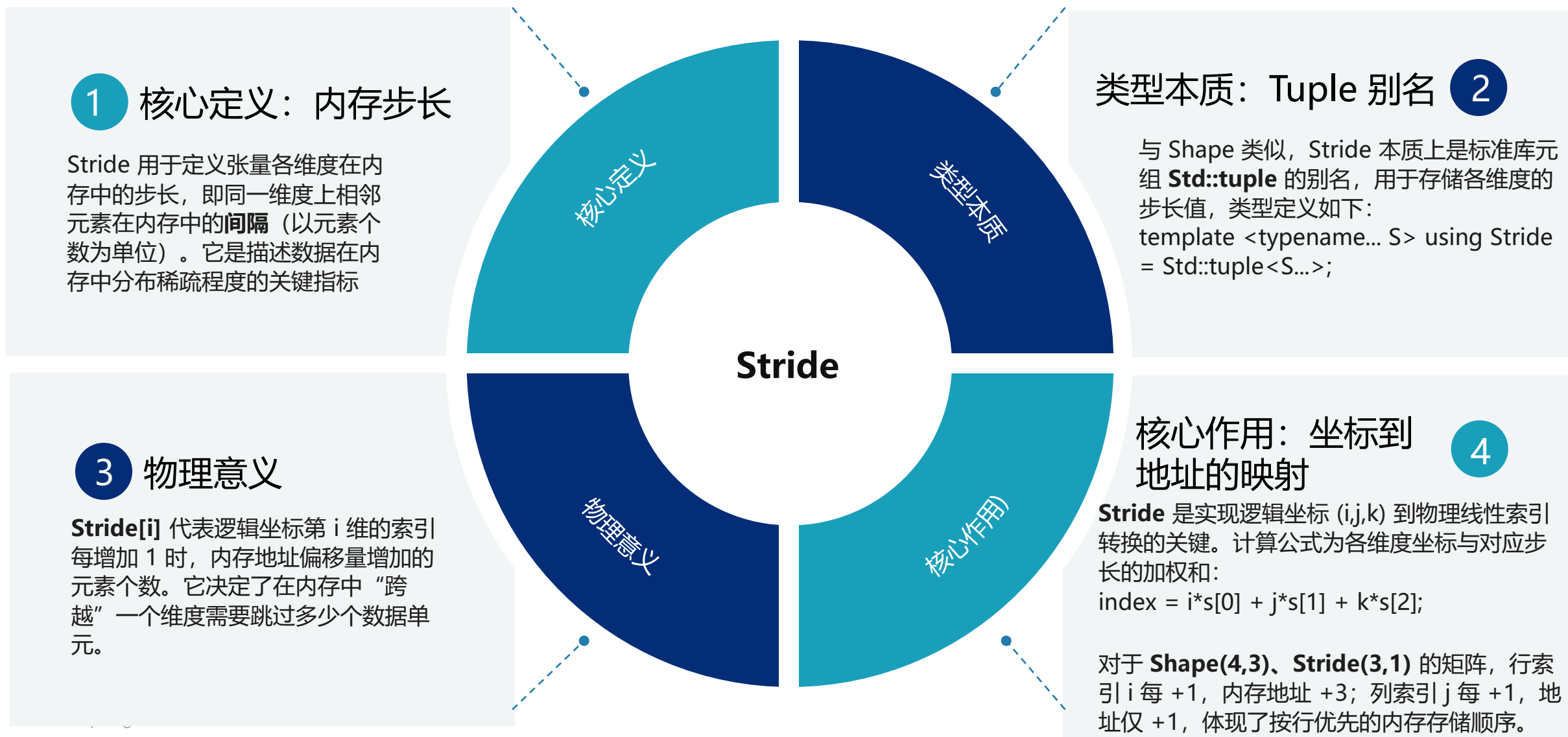
核心定义

Layout = (Shape, Stride) —— 逻辑结构与物理存储的统一映射关系

Tensor API 核心概念介绍(2)：Shape 概念解析



Tensor API 核心概念介绍(3)：Stride 概念解析



Tensor API 核心概念介绍(4)：从坐标到索引

$$\text{idx} = c_0 \cdot s_0 + c_1 \cdot s_1 + \dots + c_{n-1} \cdot s_{n-1}$$

该公式通过线性组合，将任意维度的坐标映射为连续的一维内存地址，支撑起灵活、高效的内存访问模式

逻辑坐标 ($c_0 \dots c_{n-1}$)

用户视角下的多维数据索引，代表数据在抽象结构中的位置，与物理存储无关。

映射实现

- ① 映射本质是**坐标向量**与**步长向量**的**点积 (Dot Product)**或内积运算
- ② 通过 Layout 对象的 `operator()` 运算符重载，在编译期或运行期高效完成这一映射计算。

维度步长 ($s_0 \dots s_{n-1}$)

每个维度坐标单位变化时，内存索引的增量，决定了该维度在内存中的跨度

Tensor API 核心概念介绍(5)：通用Layout示例

行优先: Shape(2, 4), Stride(4, 1)

	0	1	2	3
0	0	1	2	3
1	4	5	6	7

列优先: Shape(2, 4), Stride(1, 2)

	0	1	2	3
0	0	2	4	6
1	1	3	5	7

Layout 多层次级表达: Shape = ((2,3), (2,4)), Stride = ((1,4),(2,12))

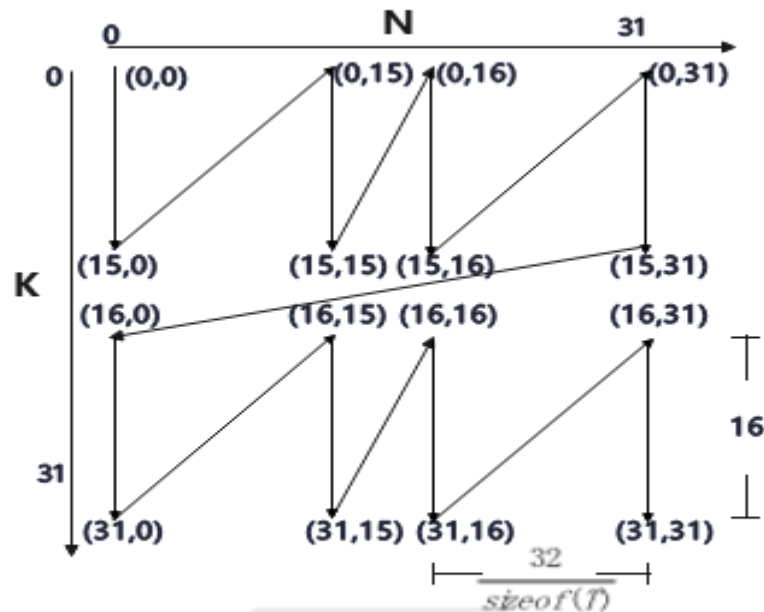
Layout = ((Shape): (Stride))
Shape = ((2, 3), (2, 4))
Stride = ((1, 4), (2, 12))

	0	1	2	3	4	5	6	7
0	0	2	12	14	24	26	36	38
1	1	3	13	15	25	27	37	39
2	4	6	16	18	28	30	40	42
3	5	7	17	19	29	31	41	43
4	8	10	20	22	32	34	44	46
5	9	11	21	23	33	35	45	47

Tensor API 核心概念介绍(6) : ZN / NZ Layout示例

ZN Layout

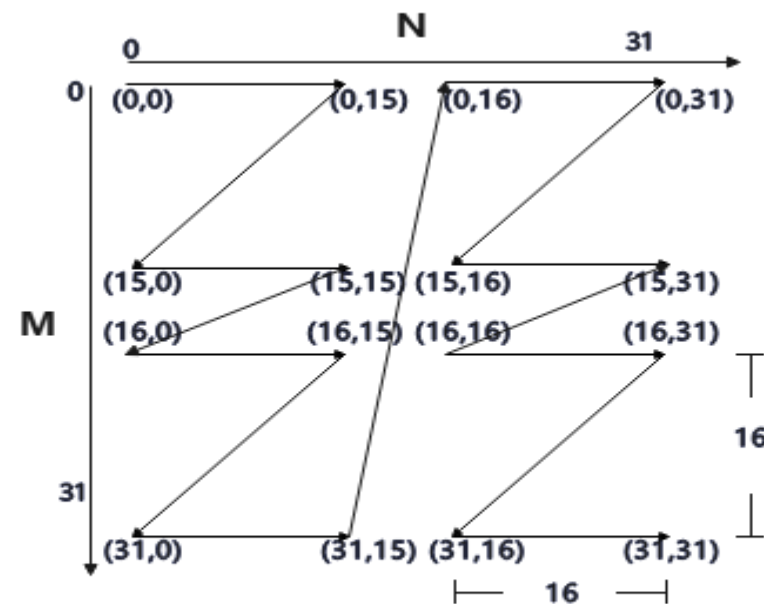
```
Layout = ((Shape) : (Stride))  
Shape = ((CO_ELEMENT, ShapeRow1), {_16},  
ShapeColumn1))  
Stride = ({_1}, StrideRow1), (CO_ELEMENT,  
StrideColumn1))  
// StrideRow1应满足:  
// StrideRow1 % (ShapeRow0 * ShapeColumn0) == 0  
// CO_ELEMENT = _32 / sizeof(T)
```



矩阵B

NZ Layout

```
Layout = ((Shape) : (Stride))  
Shape = ({_16}, ShapeRow1), {_16}, ShapeColumn1))  
Stride = ({_16}, StrideRow1), {_1}, StrideColumn1))  
// StrideColumn1应满足:  
// StrideColumn1 % (ShapeRow0 * ShapeColumn0) == 0
```



矩阵C

Tensor API 使用(1): 基于MakeLayout创建Layout

Te::MakeLayout

显式指定 Shape 和 Stride

```
auto my_shape = Te::MakeShape(8, 16);  
auto my_stride = Te::MakeStride(16, 1); // 紧凑  
行优先  
auto my_layout = Te::MakeLayout(my_shape,  
my_stride);  
手动控制步长参数, 适合需要非标准、稀疏或特定  
对齐的内存布局场景。
```

仅指定 Shape, 自动推导 Stride

```
auto my_layout =  
Te::MakeLayout(MakeShape(8, 16, 32));  
// 推导结果: Stride(16*32, 32, 1) = (512, 32, 1)  
隐式生成紧凑的行优先步长, 代码更简洁, 适用  
于绝大多数常规的、无特殊对齐要求的连续内存  
布局
```

重要提示: 通过 `Te::MakeLayout` 创建的对象仅包含基础的尺寸和步长信息, 不包含布局模式 (Pattern) 与特征 (Trait)。若需定义如分块、平铺等高级语义, 请使用专用的 `MakeFrameLayout` 函数。

Tensor API 使用(2): 基于Te::MakeTensor创建Tensor

01 核心功能: Tensor 对象工厂函数

作为创建 Tensor 对象的核心入口, 封装了底层内存与逻辑结构的绑定过程, 提供类型安全且便捷的对象实例化方式。

02 输入: 内存指针

由 MakeMemPtr 创建, 指向数据起始地址, 并标记内存区域 (GM / L1 等) 及数据类型。

03 输入: 布局结构

通过 Shape 与 Stride 描述数据的逻辑维度与内存排布, 定义 Tensor 的多维索引方式。

// 示例1: 使用GM指针和Layout创建GlobalTensor

```
auto gmPtr = AscendC::Te::MakeMemPtr<Location::GM, half>(gmAddr);  
auto gmLayout = AscendC::Te::MakeLayout(MakeShape(32, 32),  
MakeStride(32, 1));  
auto gmTensor = AscendC::Te::MakeTensor(gmPtr, gmLayout);
```

// 示例2: 使用Local Memory指针和Layout创建LocalTensor

```
constexpr int tileNum = 16 * 32;  
__cbuf__ half dataPtr[tileNum];  
auto ptr = AscendC::Te::MakeMemPtr(dataPtr);  
auto layout = AscendC::Te::MakeFrameLayout<NZLayoutPtn, half>(16, 32);  
auto tensor = AscendC::Te::MakeTensor(ptr, layout);
```

// 示例3: 使用指针和形状创建张量 (自动计算步幅)

```
auto tensor2 = AscendC::Te::MakeTensor(ptr, AscendC::Te::MakeShape(32,  
32), AscendC::Te::MakeStride(32, 32));
```


Tensor API 使用(3): Tensor 的元素访问与切片

operator[]

按多维坐标直接访问单个元素。需注意仅在全局内存(__gm__)和统一缓冲区(__ubuf__)地址空间支持, AI Core 内部存储不支持标量直接访问。

```
// 构造一个GM张量
using namespace AscendC;
auto gmLayout =
    Te::MakeLayout(Te::MakeShape(128,
128), Te::MakeStride(128, 1));
auto gmTensor =
    Te::MakeTensor(Te::MakeMemPtr<Loca
tion::GM, float>(gmAddr),
gmLayout);

// 按坐标访问元素
auto coord = Te::MakeCoord(5, 10);
auto value = gmTensor[coord];
```

operator()

获取从指定坐标起始的子张量视图。返回的子张量与原张量共享数据存储, 且保持原张量的布局结构, 仅改变数据指针的起始位置。

```
// 构造一个GM张量
using namespace AscendC;
auto gmLayout =
    Te::MakeLayout(Te::MakeShape(128,
128), Te::MakeStride(128, 1));
auto gmTensor =
    Te::MakeTensor(Te::MakeMemPtr<Loca
tion::GM, float>(gmAddr),
gmLayout);

// 获取子张量
auto subTensor =
    gmTensor(Te::MakeCoord(16,
16));
```

Slice()

切分出具有全新布局的子张量。通过指定起始坐标和目标形状, 创建一个独立布局的子张量对象, 同样与原张量共享底层数据。

```
auto gmTensor =
    Te::MakeTensor(Te::MakeMemPtr<Loca
tion::GM, float>(gmAddr),
gmLayout);

// 按Shape切片
auto sliceTensor =
    gmTensor.Slice(Te::MakeCoord(0,
0), Te::MakeShape(32, 32));
```

核心差异: operator() 根据Coord自动推导布局; Slice() 生成新布局, 适合将子区域作为独立张量处理, 二者均为零拷贝操作, 效率高。

Tensor API 使用(4): 基于Tensor的数据搬运和Mmad计算

基于MakeCopy、Copy完成数据搬运

```
using namespace AscendC;

// 定义 Copy Atom
auto copyGM2L1Atom = Te::MakeCopy(CopyGM2L1{},
CopyGM2L1TraitDefault{});

auto copyL12L0AAtom = Te::MakeCopy(CopyL12L0A{},
CopyL12L0ATraitDefault{});

// GM -> L1 数据搬运
Te::Copy(copyGM2L1Atom, l1Tensor,
gmASingleTensor.Slice(
    Te::MakeCoord(i * baseM, klter * baseK * stepK),
    Te::MakeShape(baseM, baseK * stepK))
);

// L1 -> L0A数据搬运
Te::Copy(copyL12L0AAtom, l0ATensor,
l1ATensor.Slice(
    Te::MakeCoord(0, klter % stepK * baseK),
    Te::MakeShape(curM, curK))
);
```

基于MakeMmad、Mmad完成矩阵计算

```
using namespace AscendC;

// 定义 Mmad Atom
auto mmadAtom = Te::MakeMmad(Te::MmadOperation{},
MmadTraitDefault{});

// 执行相关计算
Te::MmadParams para;
para.m = curM;
para.n = curN;
para.k = curK;
para.cmatrixInitVal = false;
if constexpr (ENABLE_UNITFLAG){
    para.unitFlag = (klter == kLoop - 1) ? 3 : 2;
}

Te::Mmad(mmadAtom.with(para),
l0CTensor, l0ATensor, l0BTensor);
```

Tensor API 使用(5): 自定义扩展支持 Atom 接口

01 新增自定义需求

自定义GM2L1的非连续数据搬运

02 新增Operator

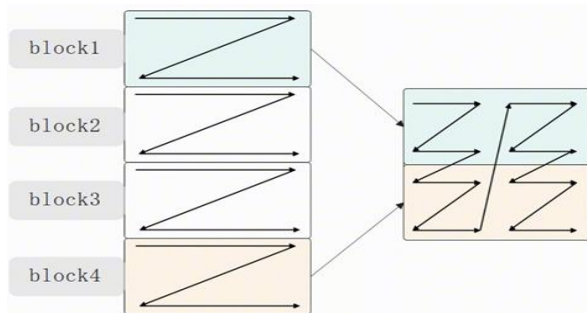
实现Operator的操作行为, 实现两块非连续的GM搬运到L1中

03 新增Trait

自定义Trait, 包含自定义参数如需要指定不同的gm地址等

04 自定义扩展Atom示例

复用MakeAtom、Copy, 快速使用CustomCopyGM2L1



```
struct CustomCopyGM2L1 {  
    template <typename Tp, const Tp&  
        traits, typename... Args>  
        __aicore__ inline static void Copy(const  
        Args&... args) {  
            // 用户使用C-API自定义实现, 根据不同的  
            // 起始地址开始计算  
            ...  
            for (uint32_t index = 0; index <  
                ADDR_COUNT; ++index) {  
                asc_copy_gm2l1_nd2nz(l0a,  
                gmAddr[index], ...);  
            }  
        }  
};
```

```
template <typename TraitStruct>  
struct CopyTraits<CustomCopyGM2L1,  
    TraitStruct> {  
    template <const TraitType& trait =  
        defaultTrait, typename... Args>  
        __aicore__ inline void CopyUnpack(const  
        Args&... args) const  
    {  
        CustomCopyGM2L1::Copy<TraitType,  
            trait, Args...>(args..., gmAddr);  
    }  
    // 自定义参数实现, 这边开发者需要指定  
    // 不同的gm地址  
    __gm__ half* gmAddr[ADDR_COUNT];  
};
```

```
auto customCopy =  
    Te::MakeCopy(CustomCopyGM2L1{});  
Te::Copy(customCopy.with( gmAddr),  
    l1Tensor, gmTensor);
```

目录

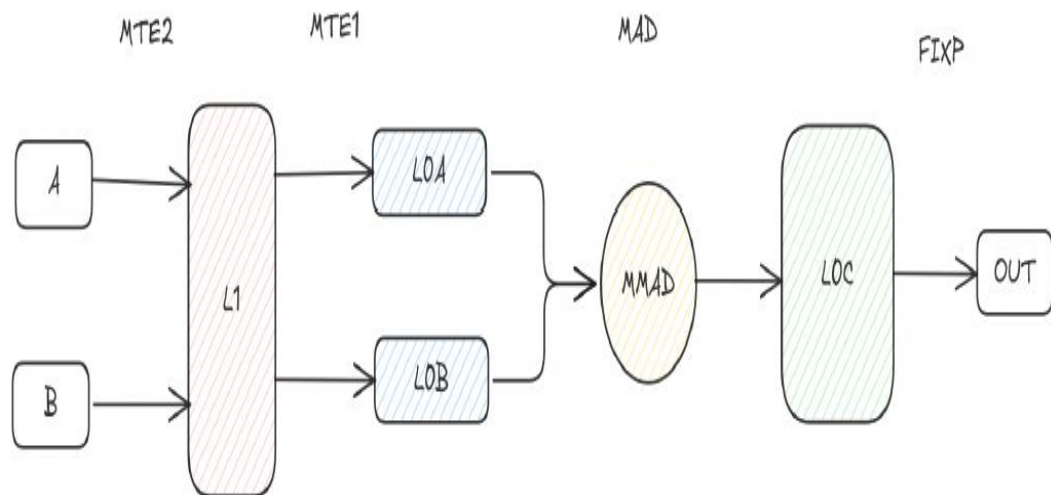
Part 1 传统Tensor编程面临的挑战和痛点

Part 2 Tensor API: Tensor编程全新升级

Part 3 基于 Tensor API 的矩阵算子开发实践

基于Tensor API的矩阵算子实践(1): Matmul算子开发的挑战

Matmul算子典型数据流



Matmul算子开发的主要挑战

- 按需使用不同的指令完成数据在不同buffer间搬运: ND2NZ/CopyPad/Load2D/Fixpipe等
- 控制循环间各个buffer的地址偏移
- 完成不同指令的参数配置

基于Tensor API的矩阵算子实践(2): Matmul算子代码样例

以B矩阵数据搬运为例，实际代码如下所示：

```
// 根据输入选择Pattern
using LayoutB = Std::conditional_t<TRANS_B,
    Std::conditional_t<IS_NZ, ZNLayoutPtn, DNExtLayoutPtn>,
    Std::conditional_t<IS_NZ, NZLayoutPtn, NDEExtLayoutPtn>>;

// 创建tensor
auto layoutB = FrameLayoutFormat<LayoutB, Std::Int<C0_SIZE>>{}(k_, n_);
auto gmB = MakeTensor(MakeMemPtr<Location::GM>(bGmAddr_), layoutB);

// 将数据从GM搬运到L1
auto layoutBL1 = MakeFrameLayout<NZLayoutPtn,
    LayoutTraitDefault<BType>>(curKL1, curN);
auto tensorBL1 = MakeTensor(MakeMemPtr<Location::L1, BType>(offset), layoutBL1);
auto gmTileB = gmB.Slice(MakeCoord(kCoord, nCoord), MakeShape(curKL1, curN));
auto copyGM2L1 = MakeCopy(CopyGM2L1{});
Te::Copy(copyGM2L1, tensorBL1, gmTileB);

// 将数据从L1搬运到L0B
auto layoutBL0 = MakeFrameLayout<ZNLayoutPtn,
    LayoutTraitDefault<BType>>(curK0, curN);
auto tensorBL0 = MakeTensor(MakeMemPtr<Location::L0B, BType>(l0bOffset),
    layoutBL0);
auto tensorTileBL1 = tensorBL1.Slice(MakeCoord(kIdx, 0), MakeShape(curK0, curN));
auto copyL12L0B = AscendC::Te::MakeCopy(AscendC::Te::CopyL12L0B{});
Copy(copyL12L0B, tensorBL0, tensorTileBL1);
```

- 不同的输入和buffer层级，只影响Layout pattern选择
- 开发者只需关注输入输出矩阵的逻辑排布(m, k)/(k, n)/(m, n)
- 只需关注各维度的逻辑坐标，无需计算物理地址偏移
- 数据搬运统一为三步：创建目的tensor、切分源tensor、调用算法接口
- 开发者不需要关心搬运指令选择和指令参数配置

基于Tensor API的矩阵算子实践(3): Tensor API优势分析

底层封装降低开发门槛 ■

基于新tensor, 合理封装内部
格式转换、地址偏移计算和指
令参数配置, 降低开发难度

降低开
发门槛

■ 接口简洁规范易用

接口简单, 统一编程范式: 构造
源tensor, 构造目的tensor, 调
用算法接口完成搬运和计算

接口简
洁

■ 统一范式缩减学习成本

合理封装和统一编程范式,
使代码结构简单, 学习成本
大大降低

降低学
习成本

■ 性能调优灵活可控

调优灵活, 优化难度降低

灵活性能调优

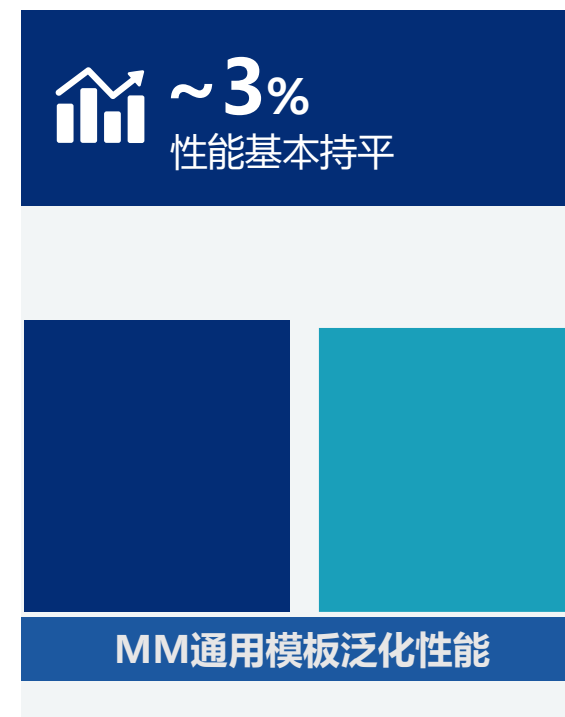
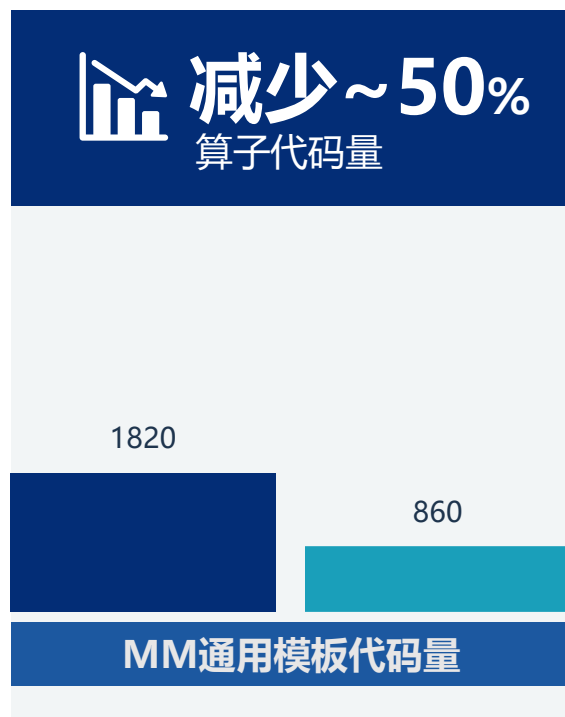
提升开
发效率

开发效率显著提升 ■

代码可读性好, 开发人员
上手快, 开发周期从4人周
降为1人周

Tensor API优
势分析

基于Tensor API的矩阵算子实践(4): Matmul算子通用模板收益



基线版本 (手工计算)

基于Tensor API实现

CANN

Ascend C Tensor API的规划

2026.6.30

首次发布，主要支持矩阵类算子开发

已支持TensorAPI的算子列表：

- Matmul
- BatchMatmul
- QuantBatchMatmul
- GroupMatmul
- WeightQuantBatchMatmul
- MC2类

2026.12.30

全量支持，并丰富更多样例

- 增加矢量类API
- 增加完善Layout代数函数
- 丰富编程指南
- 丰富更多样例

2027~

支持更多芯片

- 增加对下一代发布的芯片及时支持

附录

- Ascend C Tensor API实现与样例: <https://gitcode.com/cann/asc-devkit>
- Ascend C 编程指南: <https://asc.gitcode.com>
- CANN算子领域实战样例仓库: <https://gitcode.com/cann/cann-samples/tree/master/Samples/>
- 高性能线性代数加速引擎: <https://gitcode.com/cann/ops-tensor>

Thank you.



群聊: AscendC 算子编程交流



该二维码7天内(7月6日前)有效, 重新进入将更新

社区愿景: 打造开放易用、技术领先的AI算力新生态

社区使命: 使能开发者基于CANN社区自主研究创新, 构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯